

LAPORAN PERCOBAAN PRAKTIKUM JOBSHEET 5

BRUTE FORCE & DIVIDE CONQUER

Nama : Jonathan Abdiel Haryono
Absen : 14
Kelas : TI 1G

1. PERCOBAAN 1 : Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer

Verifikasi Hasil Percobaan

```
Masukkan nilai : 5  
Nilai faktorial 5 menggunakan BF : 120  
Nilai faktorial 5 menggunakan DC : 120
```

Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!
2. Apakah memungkinkan perulangan pada method `faktorialBF()` diubah selain menggunakan for? Buktikan!
3. Jelaskan perbedaan antara `fakto *= i;` dan `int fakto = n * faktorialDC(n-1);`!
4. Buat Kesimpulan tentang perbedaan cara kerja method `faktorialBF()` dan `faktorialDC()`!

Jawaban

1. Penggunaan if dan else adalah untuk menentukan pilihan ketika n adalah 1 dan selain 1. Jika n bukan 1, misalkan 5, maka program akan me-return variable fakto yang mana merupakan hasil kali dari n (dalam kasus ini adalah 5) dengan fungsi faktorialDC dengan parameter n-1 (dalam kasus ini adalah 5-1, yaitu 4). Kemudian, fungsi faktorialDC dengan parameter nilai 4 tadi akan terjadi hal yang sama seperti kasus pertama, yakni me-return n (dalam kasus ini adalah 4) dikali dengan fungsi faktorialDC dengan parameter n-1 (dalam kasus ini adalah 3). Begitu seterusnya hingga pada akhirnya fungsi akan me-return nilai 1 karena n mencapai 1. Sehingga, secara berurutan dari dalam, akan terjadi operasi sebagai berikut :

$$1 * 2 * 3 * 4 * 5 = 120$$

2. Memungkinkan, dengan do while :

```
int fakto = 1, i = 1;

do {

    fakto *= i;

    i++;

} while (i <= n);

return fakto;
```

dengan while :

```
int fakto = 1, i = 1;

while (i <= n) {

    fakto *= i;

    i++;

}

return fakto;
```

3. `fakto *= i` artinya mengalikan fakto dengan i dan memasukkannya pada fakto yang baru. Sedangkan, `int fakto = n * faktorialDC(n - 1);` artinya mengalikan n dengan fungsi `faktorialDC(n-1)` dan memasukkannya pada fakto.

Maknanya sama, yakni mengalikan sederet bilangan dari 1 hingga n (termasuk n), hanya saja menggunakan metode yang berbeda

4. Cara kerja `faktorialBF` adalah mengalikan angka dari 1 hingga n secara berurutan. Sedangkan, `faktorialDC` bekerja dengan cara mengalikan n dengan fungsi `faktorialDC` itu sendiri dengan parameter `n-1` seterusnya hingga `n = 1` dan mengembalikan nilai secara berurutan $5 * 4 * 3 * 2 * 1$, sehingga mendapatkan nilai 120, sama dengan `faktorialBF`.

2. PERCOBAAN 2 : Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer

Verifikasi Hasil Percobaan

```
Masukkan jumlah elemen : 3
Masukkan nilai basis elemen ke-1 : 2
Masukkan nilai pangkat elemen ke-1 : 3
Masukkan nilai basis elemen ke-2 : 4
Masukkan nilai pangkat elemen ke-2 : 5
Masukkan nilai basis elemen ke-3 : 6
Masukkan nilai pangkat elemen ke-3 : 7
HASIL PANGKAT BRUTEFORCE :
2^3 : 8
4^5 : 1024
6^7 : 279936
HASIL PANGKAT DIVIDE AND CONQUER :
2^3 : 8
4^5 : 1024
6^7 : 279936
```

Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu `pangkatBF()` dan `pangkatDC()` !
2. Apakah tahap *combine* sudah termasuk dalam kode tersebut? Tunjukkan!
3. Pada method `pangkatBF()` terdapat parameter untuk melewati nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class `Pangkat` telah ada atribut `nilai` dan `pangkat`, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method `pangkatBF()` yang tanpa parameter?
4. Tarik tentang cara kerja method `pangkatBF()` dan `pangkatDC()` !

Jawaban

1. `pangkatBF()` menggunakan metode brute force, yakni dengan melakukan perulangan terhadap variabel hasil dikali dengan a (yakni basis) sebanyak n (yakni pangkat) kali. Sedangkan, `pangkatDC()` menggunakan metode divide & conquer, yakni dengan memecah operasi perkalian berulang menjadi beberapa bagian operasi hingga bagian terkecilnya, yakni pangkat 1.
2. Sudah, dengan adanya operasi perkalian antar bagian, secara otomatis, program sudah melakukan tahap combine, yakni menghubungkan tiap bagian dengan bagian lainnya dengan operasi perkalian.
3. Menurut saya, method tersebut masih relevan, karena penggunaannya dapat mempermudah dan mempersingkat dalam menulis program. Method memang bisa dibuat

tanpa parameter, tetapi akan memerlukan langkah ekstra apabila tidak menggunakan parameter, yakni inisialisasi nilai atribut satu per satu pada atribut yang akan digunakan.

Method pangkatBF() tanpa parameter adalah sebagai berikut :

```
int pangkatBF () {  
    int hasil = 1;  
    for (int i = 0; i < pangkat; i++) {  
        hasil = hasil * nilai;  
    }  
    return hasil;  
}
```

4. Method pangkatBF() melakukan perulangan sebanyak n (pangkat) kali terhadap perkalian variabel hasil dengan a dan memasukkannya ke variabel hasil yang baru. Begitu seterusnya hingga mendapat nilai hasil. Sedangkan, method pangkatDC(), pertama, melakukan pemilihan apakah n adalah 1 atau tidak. Jika bukan 1, program akan melakukan pemilihan lagi apakah n (iterasi pangkat) genap atau ganjil. Jika n ganjil, maka program membagi nilai kembalian menjadi 3 bagian, yakni 2 buah fungsi rekursif dengan parameter a, n/2 dan 1 buah nilai a dengan jembatan operasi perkalian. Jika n genap, maka program hanya akan membagi nilai kembalian menjadi 2 bagian, yakni 2 buah fungsi rekursif dengan parameter a, n/2 dengan jembatan operasi perkalian. Sehingga, pada tiap bagian terkecil, akan terjadi perkalian dari angka pangkat 1 dengan angka pangkat 1 dan dengan angka a (yakni nilai yang dipangkat) jika ganjil.

3. PERCOBAAN 3 : Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer

Verifikasi Hasil Percobaan

```
Masukkan jumlah elemen : 5  
Masukkan keuntungan ke-1 : 10  
Masukkan keuntungan ke-2 : 20  
Masukkan keuntungan ke-3 : 30  
Masukkan keuntungan ke-4 : 40  
Masukkan keuntungan ke-5 : 50  
Total keuntungan menggunakan Brute force : 150.0  
Total keuntungan menggunakan Divide and Conquer : 150.0
```

Pertanyaan

1. Kenapa dibutuhkan variable `mid` pada method `TotalDC()` ?
2. Untuk apakah statement di bawah ini dilakukan dalam `TotalDC()` ?

```
double lsum = totalDC(arr, l, mid);  
double rsum = totalDC(arr, mid+1, r);
```
3. Kenapa diperlukan penjumlahan hasil `lsum` dan `rsum` seperti di bawah ini?

```
return lsum+rsum;
```
4. Apakah base case dari `totalDC()` ?
5. Tarik Kesimpulan tentang cara kerja `totalDC()`

Jawaban

1. Variabel `mid` dibutuhkan sebagai pembagi masalah menjadi 2 sehingga program bisa mendapatkan index tengah dari suatu array.
2. Untuk memanggil fungsi kembali, namun dengan parameter setengah dari panjang array sebelah kiri disimpan dalam variabel `lsum` dan setengahnya lagi yang sebelah kanan disimpan dalam variabel `rsum`.
3. Karena, pada akhir rekursi, diperlukan penjumlahan, baik dari sisi kiri maupun kanan, yakni nilai `lsum` dan `rsum` sebagai nilai total dari sebuah array.
4. Base case dari `totalDC()` adalah ketika nilai `l` (index kiri, left) sama dengan nilai `r` (index kanan, right), sehingga program akan me-return array dengan index nilai tersebut, `arr[l]` atau `arr[r]`, keduanya sama saja karena `l` sama dengan `r`.
5. Cara kerja method `totalDC()` adalah dengan membagi sebuah array menjadi beberapa bagian menggunakan 2 buah pointer hingga kedua pointer tersebut menunjuk pada index yang sama pada sebuah array. Ketika hal itu terjadi, program akan mengembalikan nilai array pada index tersebut (`return arr[l]` atau `return arr[r]`) dan menyimpannya pada `lsum` (untuk sisi kiri) dan `rsum` (untuk sisi kanan). Pada akhirnya, program akan mengembalikan nilai total dari `lsum` dan `rsum`.